

one

Bash

Terminal & Finder

Terminal works as a **REPL**

- R : Read
- E : Eval
- P : Print
- L : Loop

Some Commands

```
echo
```

```
pwd
```

```
ls
```

```
touch file.txt -> Make a file
```

```
rm file.txt -> remove a file
```

```
clear
```

```
cd
```

File Manipulation

```
> touch hello.txt
```

```
> touch hello12.txt
```

```
> mv hello.txt hello1.txt
```

```
> mv hello1.txt hello12.txt
```

```
#mv command will rename or move without giving any errors
```

```
#4th command works without giving any errors
```

```
> rm *.txt
```

```
rm -i #for interactive mode -> asks you what to remove -> `makes  
it less dangerous`
```

```
alias rm='rm -i'
```

history -> get history of the bash

hidden files

- file that start with . is a hidden file
- `ls -a` -> gives you all the files
- `'.'` -> current directory
- `'..'` -> directory one level above
- `'-'` -> previous directory

```
cd downloads
cd ~/documents
cd - # gets you back into in the downloads
```

searching in files

`cat` -> open your files

concat

```
cat hello.txt
#everything inside the hello.txt is printed to the terminal
```

grep -> another tool which is used for the finding patterns in a input stream or file

```
grep 'hello' /usr/share/dict/words
#everything having 'hello' in it

grep '^hello' /usr/share/dict/words
#everything starting with 'hello' in it

grep 'hello$' /usr/share/dict/words
#everything ending with 'hello' in it
```

```
echo hello > file.txt
#overwrites the whole file and write
```

```
echo hello >> file.txt
#appends to the file.txt
```

```
grep -A1 hello file.txt
#find hello and get me the one line after it
```

```
grep -B1 hello file.txt
#find hello and get me the one line before it
```

```
grep -C1 hello file.txt #C:-> context
#find hello and get me the one line before and after it
```

```
grep -i hello file.txt #i:-> insensitive
#find hello irrespective of the case
```

```
grep -o hello file.txt #o:-> only the part that has matched the
pattern
```

```
#we can also combine these
grep -io hello file.txt
grep -oi hello file.txt
grep -i -o hello file.txt
grep -o -i hello file.txt
## all of these are the same things
```

```
#use grep to get the count of the words
grep -c d /usr/share/dict
```

pipelines

```
cat file.txt | grep hello
#if we don't provide a input grep reads from its standard input
#all the data of the file.txt is parsed to grep and grep
searches for hello it is the same as `grep hello file.txt`
```

paging files

making a large output of a command readable

- 'less'
- 'more'

less is more powerful than **more**

```
less cat /usr/share/dict/word
more cat /usr/share/dict/word
#these do the same thing

cat /usr/share/dict/word | less
#piping to the less
```

man pages

manual pages

```
man mv
# manual for the mv
# three important sections
# 1 :- General commands `mv` `cp` `ls`
# 2 :- System Calls Manual
# 3 :- Library Functions manual `std::vector`

man 1 cp
man printf -> printf for the terminal
man 3 printf -> printf from the C
```

- man equivalent **history**

```
help history #-> for the things built into the shell
```

- type command

```
type history
> history is a shell builtin

type -a ls
# everything from the top to bottom
# alias ls='ls -p --color=auto'
# ls
# so the type command would run all the options or the alias
before going to bed
```

We need to use help command for the built in types

Programs and commands

`file` -> tells us the type of the program

Q) when you typed 'ls' how did the bash know where to look?

ans: it was already mentioned in the path

`echo $PATH` we used \$ in-order to instruct the bash to expand the variable

```
echo $PATH

echo $PATH | tr : '\n'
#tr -> translate command
#changes the ':' -> '\n'
```

tr command

```
echo "$PATH" | tr : '\n' | tr 'a' 'b'
# replace ':' -> tr and 'a' -> 'b'

echo "$PATH" | tr :a '\nb'
# same as the first one

echo "$PATH" | tr abc 'xyz'
```

```
# a -> x
# b -> y
# c -> z
```

some more variables

Use `$` to expand these

- PWD
- USER
- SHELL
- HOSTNAME
- MACHTYPE -> machine type
- uname

defining a variable

```
name='Medhansh'

#if i want to have a dyanaminc name
name=`uname -a`
#C equivalent printf("%d");
#this is deprycated so we use this

name=$(uname -a)
```

Vim the editor

commands to activate/open

1. vi
2. vim
3. nvim

```
sudo dnf install nvim
sudo apt install nvim
```

to open `vim file.txt`

three modes of vim

1. normal
2. visual
3. editing

1. normal

to enter this mode

comes by default (When you open the file)

`dd` -> removes the line (no matter where the cursor is)

`.` -> replays the last command

`p` -> paste

`yy` -> yanking (copying)

`u` -> undo

`G` -> end of the file (shift + g)

`gg` -> top of the file

2. insert

to enter this mode use the following after entering the normal mode

1. `o` (start editing Below the line where the cursor is)
2. `i` (start editing where the cursor is right now)
3. `O` (start editing Above the line where the cursor is)
4. `I` (start editing from the starting of the line)

Use `Esc` to exit a mode

now to execute any command come back to normal mode and press

`:` to execute

Scripting

Bash script is just a series of commands

```
vim script.sh
name='medhansh'
number=12
system=$(uname)

#remember echo is just printf
echo "hello $name (number is $number)"

> output :-> hello medhansh (number is 12)
```

file permissions

- `ls -l` !:-> long listing
- now to change the permission of a file use `chmod`

now this command

```
chmod +x script.sh
# now +x means add executable permission

chmod 755 script.sh
# same as above
```

now here comes the main thing the one and only SHEBANG

```
#!/bin/bash
```

- This tells the system about what compiler should be the file executed from
- Or better one :- Which language i will be speaking for the rest of the file (English, dutch, esponal)

scripting

First Progame (Hello, World!)


```
vim hello
    echo "Hello, World!"
:wq

chmod +x hello
./hello
> Hello, World!
```

Variable expansion

```
vim expansion
    name='Medhansh'
    echo "Hello, $name!"
:wq

chmod +x expansion
./expansion
> Hello, Medhansh!
```

for loops

- for loops are something like iterating over a array
- something like javascript

```
vim for
    for var in med han sh jo sh; do
        echo "var is $var"
    done

chmod +x for
./for
> var is med
> var is han
> var is sh
> var is jo
> var is sh
```

syntax checker `bash -n`

used to check the syntax of your script

```
bash -n for
# no errors -> nothing is displayed
# to check exit code of the last command use

echo $?

0          -> worked like a charm
non-zero  -> these is an issue
```

taking input

`read` is the command

```
vim input
    read -p "Enter your name: " name
    echo "Hello, $name"

chmod +x input
./input
> Enter your name : Medhansh
> Hello, Medhansh
```

- a trick `yes`
- `yes` is a valid command in your system use it when you want to say yes multiple times and you are aware of the steps

```
yes | ./input

> Hello, y
```

argc and argv

- count of the variables argc -> is stored in the variable #
- argv the array of the words -> is stored in the variable @

```
vim input
#!/usr/bin/env bash
# read -p "Enter your name: " name
echo "Hello, $1"
```

```
chmod +x input
./input Medhansh
```

```
> Hello, Medhansh
```

```
vim input
#!/usr/bin/env bash
if [[ -n $1 ]]; then
    name = $1 #if something is there in name || the name is
not empty
else
    read -p "Enter your name: " name
    #if name comes out to be empty
fi #this is how to close the

echo "Hello, $1"
```

```
chmod +x input
```

```
./input Medhansh
> Hello, Medhansh
```

```
./input
> Enter your name: Medhansh
> Hello, Medhansh
```

```
help '['
help test
```

@ operator

```
vim for
  for var in "$@"; do
    echo "var is $var"
  done
```

```
chmod +x for
.for med han sh jo sh
> var is med
> var is han
> var is sh
> var is jo
> var is sh
```

```
.for med han sh
> var is med
> var is han
> var is sh
```

```
.for med han
> var is med
> var is han
```

```
.for med
> var is med
```

@ -> this is expand until there is no more left

Functions

```
hello(){
  name = $1
  # $1 -> the first argument passed to the function
  #the name declared like this becomes a global variable
  so instead we use
```

```
}
```

```
hello(){  
    local name = $1  
    echo "Hello, $name"  
    return 0  
}t
```

Conditionals

```
#!/usr/bin/env bash  
a=2  
b=2  
  
if [[ $a == $b ]]; then  
    echo "a and b are equal"  
fi  
  
c=2  
d=3  
  
if [[ $a != $b ]]; then  
    echo "c and d are not equal"  
fi  
  
> a and b are equal  
> c and d are not equal
```

checking in files **-f**

```
if [[ -f file.txt ]]; then  
    echo 'file.txt is a file and it exists'  
fi  
  
> 'file.txt is a file and it exists'
```

TO get more about the flags use `help test`

a bit of *while* loop

```
while [[ -f file.txt ]]; do
    echo 'File.txt exists and is a file'
    sleep 1
done
## till file exists it will print file.txt exists and is a file
## sleep 1 -> delay(1)
```

a bit about *until* loops

- until is opposite of while loops
- rarely used

```
until [[ -f file.txt ]]; do
    echo 'File.txt does exists'
    sleep 1
done
## untill file doesn't exists it will print 'file.txt doesn't
exists'
## sleep 1 -> delay(1)
```

using with commands

```
if ls; then
    echo 'ls worked'
fi

> all the files in the dir
> ls worked
```

true & false

```
if true ; then
    echo "it is true"
```

```
else
    echo "it is false"
fi
```

BackThoughts

```
if apt-get update; then
    echo 'syatem Upadted'
else
    echo 'not able to update the system'
fi
```

For loops

```
#!/usr/bin/env bash

for name in med han sh; do
    echo "name is $name"
done
```

{} notation

- {} -> Range in python

```
#!/usr/bin/env bash

for thing in {a..f}; do
    echo "this is $thing"
done
```

```
>this is a
>this is b
>this is c
>this is d
>this is e
>this is f
```

```
#!/usr/bin/env bash
```

```
for thing in {1..6}; do  
    echo "this is $thing"  
done
```

```
>this is 1  
>this is 2  
>this is 3  
>this is 4  
>this is 5  
>this is 6
```

C like for loops

```
max=5  
for((i =0; i<max;i++)); do  
    echo "thing is $i"  
done
```

```
>this is 0  
>this is 1  
>this is 2  
>this is 3  
>this is 4
```

input and output (I/O)

```
#!/usr/bin/env bash  
read fo  
echo "you said $fo"
```

this script above can cause errors

this won't keep data raw

```
You entered : 'hello \n world'  
displayed : 'hello n world'
```


-r flag

so instead use this flag -r (raw)

```
#!/usr/bin/env bash
read -r fo
echo "you said $fo"
```

- line by line reading in bash is slow

regex

general expressions

for example

emails :- are like name@domain

now `name` and `domain` are variable here

now

`a*` -> a word that has a starting character a -> `'a'kriti`

`m*` -> a word that has a starting character m -> `'m'edhansh`

```
#name="medhansh"
read -pr "enter you name" name
if [[ $name == m* ]]; then
    echo "you are the great, $name"
elif [[ $name == j* ]]; then
    echo "you are not as great as M, $name"
else
    echo "No need of yours"
fi
```

> enter your name : medhansh

> you are the great, medhansh

> enter your name : jayant

> you are not as great as M, jayant

```
> enter your name : rehan  
> No need you yours
```

case statements

```
s = $1  
  
case "$s" in  
    medhansh)  
        echo hi medhansh  
        ;;  
    jayant)  
        echo hi jayant  
        ;;  
    rehan)  
        echo so ja  
        ;;  
    *)  
        echo hello world  
esac
```

```
> ./case medhansh  
> hi medhansh
```

```
> ./case jayant  
hi jayant
```

- `;;` -> this acts like a `break;`
- `*` -> default case
- `&` -> this is for when you want default behaviour just like in C language

Classic C behaviour

```
s='medhansh'
```

```
case "$s" in
```

```
m*) echo 'matched m*';&
medhansh) echo 'matched medhansh';&
f*) echo 'matched f*';&
cati) echo 'matched cati';&
*) echo 'matched *';&
esac

> matched m*
> matched medhansh
> matched f*
> matched cati
> matched *
```

Typical Behaviour

```
s='medhansh'

case "$s" in
    m*) echo 'matched m*';;
    medhansh) echo 'matched medhansh';;
    f*) echo 'matched f*';;
    cati) echo 'matched cati';;
    *) echo 'matched *';;
esac

> matched m*
```

New kind of switch-case

`;;&` -> checks every case if matches then does the action

```
s='medhansh'

case "$s" in
    m*) echo 'matched m*';;&
    medhansh) echo 'matched medhansh';;&
    f*) echo 'matched f*';;&
    cati) echo 'matched cati';;&
```

```
*) echo 'matched *';;&
esac

> matched m*
> matched medhansh
> matched *
```

Indexed-Array

```
array=(med han sh jo)

echo "${array[0]}"
echo "${array[1]}"
echo "${array[2]}"
echo "${array[3]}"
echo "${array[4]}"

> med
> han
> sh
> jo
>
#bash will treat it like an empty
```

-ve indices

```
array=(med han sh jo)

echo "${array[-1]}"
echo "${array[-2]}"
echo "${array[-3]}"
echo "${array[-4]}"

> jo
> sh
```

```
> han
> med
```

indices using variable

```
idx=2

array=(med han sh jo)

echo "${array[$idx]}"
echo "${array[idx]}"
#these both syntax are valid in bash
```

@ && * in bash

```
array=(med han sh jo)

for i in "${array[*]}"; do
    echo "item is $i"
done

> med han sh jo
```

```
array=(med han sh jo)

for i in "${array[@]}"; do
    echo "item is $i"
done

> med
> han
> sh
> jo
```

- * -> string-ifies the array (aka converts array into the string)
- @ -> treat it like a normal array

copying an array and add elements



```
first_array=(
    med
    han
    sh
    jo
)

second_array=("${first_array[@]}")

second_array+=(sh i)

for i in "${second_array[@]}; do
    echo "item is : $i"
done
```

- `echo $array` gives the first element of the array

Another way to declare array (Sparse arrays)

```
array=( [0]="med" [1]="han" [2]="sh" )

#why is this used

array=( [0]="med" [1]="han" [2]="sh" [23]="jo" )
```

- get the length of the array
`echo "${#array[@]}"`
- Get the length of the string
`echo "${#array[23]}"` -> length of the word at 23rd index

Associative Array (These are relatively new)

- Something like hash Map
- We need to use declare them using `declare`

```
#!/usr/bin/env bash
```

```
declare -A arr
```

```
arr[med]=11
```

```
arr[han]=23
```

```
arr[sh]=9
```

```
echo "${arr[med]}"
```

```
> 11
```

`!` -> in-direction operator

getting a value in Associative array

- Spoiler alert we will use `!`

```
#!/usr/bin/env bash
```

```
declare -A arr
```

```
arr[med]=11
```

```
arr[han]=23
```

```
arr[sh]=9
```

```
echo "${!arr[*]}"
```

```
> med han sh
#remember * operator will stringi-fies
```

IFS variable

- built-in to bash
- it tell how to stringi-fy the array (from which character)

```
array=(med han sh)

echo "array is ${array[*]}"

> med han sh
```

- unset variables using `unset variable-name`

```
array=(med han sh)

IFS=,
echo "array is ${array[*]}"

> med,han,sh
```

Command Substitution

- running a command in bash and storing its output
- we need to use backticks

```
thing=`whoami`
echo thing
```

- nesting them becomes a problem

better way to substitute the commands (dollar-pram)


```
thing="$(whoami)"
echo "$thing -> $(uname)"

> medhansh -> linux
```

-> using these they run commands in a different subshell

```
i=5
funct(){
    echo "i am $1"
    i=6
}

echo "$(funct hello)"
echo "i is $i"

> i am hello
> i is 5
# all the change it made inside the function is dropped once the
function is over
```

- these notation can't modify the global scope

another syntax for the substitution (GLOBAL one)

- introduced in 2025 bash 5.3

```
i=5
funct(){
    echo "i am $1"
    i=6
}

echo "${ funct hello; }"
echo "i is $i"
```

```
> i am hello  
> i is 6
```

- this will change the variable
- it didn't run in its own subshell
- hence making it fast as no new subshell was made

Arithmetic Expressions

```
thing=$(( 12+34 ))  
echo $thing  
  
> 46
```

- `**` -> exponent
- `*`
- `/`
- `-`
- `+`

```
q=1  
w=2  
  
echo $(( $q + $w ))
```

```
q=1  
w=2  
  
echo $(( q + w ))
```

Some more

- `++i`
- `i++`
- `i *= 3`

- | | << left shit and right shift
- ternary

using ternary

```
a=12
b=2
((max = a > b ? a:b))

echo $max
```

((....)) this thing has a return code too

- 0 (if passed or the answer is something)
- 1 (failed or the answer was 0)

```
a=05
echo $a
echo $(( a ))
```

```
> 05
> 5
```

```
a=06
echo $a
echo $(( a ))
```

```
> 06
> 6
```

```
a=07
echo $a
echo $(( a ))
```

```
> 07
> 7
```

```
a=08
echo $a
echo $(( a ))

> error
```

- using zero in-front bash treats it like an octal number (base 8)
- we need to be careful when using it

Forcing into a specific base

- using `#`

```
a=08
echo $a
echo $(( 10#$a ))

> 08
> 8
```

Process substitution

- to get the count of the all the words

```
grep -c d /usr/share/dict
```

```
#!/usr/bin/env bash
words=$(grep d /usr/share/dict/words)
I=0
for words in $words;do
    echo $word
    ((I++))
done

echo "count is : $I"
```

- now the issue with this code it that it is waiting for command to run and then do the processing

- what if the words were 90M or more

Here Strings

```
#!/usr/bin/env bash
words=$(grep d /usr/share/dict/words)
I=0
while read -r words;do
    echo $word
    ((I++))
done <<< "$words"

echo "count is : $I"
```

- same issue as above
- first is processing and then comes out thing
- what about doing this in a streaming fashion

Another type of syntax to go with

```
#!/usr/bin/env bash
I=0
grep d /usr/share/dict/words | while read -r words;do
    echo $word
    ((I++))
done

echo "count is : $I"
```

- the answer will be correct because the command right to the pipe runs in a sub-shell

Finally command substitution

- remember about the `<` this was used way back to read and write from a file (mostly read for writing the other one is used)

```
#!/usr/bin/env bash
```

```
I=0
```

```
while read -r words;do
```

```
    echo $word
```

```
    ((I++))
```

```
done< <(grep d /usr/share/dict/words)
```

```
echo "count is : $I"
```

- now the while loop and the finding work on the same time hence increasing your script speed when working on something important
- the issue with this is
 - what is the return code (how do i get to know the command ran as usual)
 - the answer is : you don't you need to be sure about what you are trying to access (validate the input/data before processing)
 - in short (technical) it will mask the exit codes of the commands so be AWARE
- dissecting the script even further
 - now see this syntax `done< <( . . . . . )`
 - now remember the first line i wrote before starting this session
 - this grep command has to return in some sort of file for processing
 - *so when ever trying to use in future you need to make sure the command does return a file*
- some more examples for better understanding
- `echo $(uname)` we discussed this and it gave the name of your system
- `echo <(uname)` -> from the above logic it should return something file type
 - it returns a file descriptor
- `cat <(uname)` -> remember cat was used to read files
 - it will give the output you were hoping for

SUMMARY HERE

TREATING IT LIKE A FILE

cut && **tr**

```
echo $PAST
```

```
/home/medhansh/.local/bin:/usr/bin:/usr/local/sbin:/usr/local/bin:/usr/bin:/var/lib/flatpak/exports/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl:/home/medhansh/.local/bin:/home/medhansh/.local/bin
```

now we'll add the operations on this

```
echo $PAST | tr : '\n'
```

-----OUTPUT-----

```
/home/medhansh/.local/bin
/usr/bin
/usr/local/sbin
/usr/local/bin
/usr/bin
/var/lib/flatpak/exports/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
/home/medhansh/.local/bin
/home/medhansh/.local/bin
```

```
cat hello.csv | cut -d , 1-
```

- -d tells the cut - were to break it from
- 1- means till the end of the columns

sed , **awk** && **grep**

The Hunter (GREP)

```
grep "medhansh" name.txt
```

-i flag

```
grep -i "medhansh" name.txt
```

- ignores the case `medhansh` `Medhansh` `medhAnsh` `medHansh`
- all the names are matched

-r flag

```
grep -r "medhansh" .
```

- search in the current directory
- don't stop after searching one file

-n flag

```
grep -n "medhansh" name.txt
```

- show the line numbers too

-v flag

```
grep -v "medhansh" name.txt
```

- exclude medhansh from the result
- or work opposite of the grep

-c flag

```
grep -c "medhansh" name.txt
```

- number of times medhansh has been mentioned in name.txt

regex

start

```
grep "^medhansh" name.txt
```

- all the line starting with medhansh
(i know all the lines 🤖)

end

```
grep "medhansh$" name.txt
```

- all the lines ending with medhansh

lines

```
grep "[0-9]" name.txt
```

- all the lines containing numbers

SED (stream editor)

-transforms into something

FILE rn

```
medhansh  
rehan  
neeraj
```

```
sed 's/medh/ansh' name.txt
```

```
ansh  
rehan  
neeraj
```

-i flag

- inplace
- make the change in a file rather than a stream

```
sed -i 's/medh/ansh/g' name.txt
```

delete a line

```
sed '3d' name.txt
```

- delete line number 3

```
sed '/rehan/d' name.txt
```

- deletes all the lines with the word rehan in them

replacement

```
sed '2s/apple/orange/' file.txt
```

- replace the secon line

awk

- it the one of the extensive and the useful tool
- a new language in itself

print a specific column

```
awk '{print $1}' name.txt
```

```
awk '{print $1 $3}' name.txt
```

the one and only the BEAST the POWER the INSPIRATION printf

```
awk '{printf("first name is %s, and the last is %s", $1,$2)}'  
name.txt
```

change the delimiter -F

- same thing as IFS but opposite for work

- IFS were for joining based on it and
- delimited is the character to find where to start breaking from

```
awk -F "," '{print $1}' name.txt
```

conditionals

```
awk '$3 > 50 {print $1}' marks.txt
```

built-in variables

variable name	notation
\$0	Entire line
\$1	First column
NF	Number of fields
NR	Line Number

Sort and Uniq

sort

- sorts by-default on the Alphabets
- piping is the easiest option

```
stream | sort
```

uniq

- the string/stream should be sorted first hand to use uniq

using combined

```
stream | sort | uniq -c
```

- -c for the count

- it makes a basic frequency counter

Word Count (wc)

- useful for counting the raw bytes
- don't work for emojis 🙅

```
/usr/share/dict/words | grep m | wc
```

-l -> counts lines

-w -> count words

-c -> counts characters

Find

- useful for finding the files

```
find <starting-point> <conditions> <actions>
```

```
find . -name "*.txt"
```

- Start at `.`
- For each file
- If name matches `*.txt` → print it.
- printing is default action